Geração de código Intermediário
MEPA: parte 2 (Programa Completo)

Fabio Lubacheski
fabio.lubacheski@mackenzie.br

Comandos condicionais

Considere o trecho de código em **`Pascal`**:

```
....
a := 1;
b := 2;
if a > b then
   maior := a
else
   maior := b;
....
```

Como traduzir o trecho acima ?

Comandos condicionais

Para **simplificar** a tradução nos exemplos com comandos condicionais iterativos utilizaremos algumas convenções:

- **nomes simbólicos** em vez de endereço de variáveis na memória `M[]`.
- **rótulos simbólicos** no lugar endereços na memória `P[]`.
- a instrução **`NADA`** que não tem efeito sobre a execução do processo tradução, servindo somente para marcar desvio de código na memória P[].

  `NADA (Não faz nada)`

Adotamos a convenção de representar os valores booleanos por inteiros: **`true`** por **`1`** e **`false`** por **`0`**.

Instruções lógicas e relacionais

IMPORTANTE: A partir dessas instruções omitiremos na descrição das instruções a operação **`i <- i + 1`** que está **implícita** em todas as instruções, exceto quando há desvios.

```
CONJ (E logico)
    se M[s-1] = 1 E  M[s] = 1 entao
      M[s-1] <- 1
    senao
      M[s-1] <- 0
    fim-se
    s <- s – 1
```

```
DISJ (OU logico)
    se M[s-1] = 1  OU  M[s] = 1 entao
      M[s-1] <- 1
    senao
      M[s-1] <- 0
    fim-se
    s <- s – 1
```

Instruções lógicas e relacionais

```
CMME (comparar menor) <
    se M[s-1] < M[s] entao
      M[s-1] <- 1
    senao
      M[s-1] <- 0
    fim-se
    s <- s – 1

CMMA (comparar maior) >
    se M[s-1] > M[s]  entao
      M[s-1] <- 1
    senao
      M[s-1] <- 0
    fim-se
    s <- s – 1
```

```
CMIG (comparar igual) =

CMDG (comparar desigual) /=

CMEG (comparar menor igual) <=

CMAG (comparar maior igual) >=
```

Instruções de desvio em MEPA

Os desvios condicionais e incondicionais são representados pelas instruções:

```
DSVS p (Desviar sempre)
    i <- p

DSVF p (Desviar se falso)
    se M[s] = 0 entao
       i <- p
    senao
      i <- i + 1
    fim-se
    s <- s – 1
```

Onde **p** é um inteiro que indica um endereço do programa na memória **P[]**.

Comandos condicionais

Poderíamos generalizar um comando condicional conforme abaixo, onde E é uma expressão lógica e C_1 e C_2 são comandos.

```
....
if E then
   C1
else
   C2
....
```

A tradução ficaria da seguinte forma:

```
        .
        .
        .   } tradução de E
    DSVF L1
        .
        .
        .   } tradução de C1
    DSVS L2
L1:NADA
        .
        .
        .   } tradução de C2
L2:NADA
```

Exercício

1) Faça a tradução do trecho abaixo em **`Pascal`** para instruções da linguagem MEPA **utilizando nomes de variáveis e rótulos simbólicos**.

```
....
if a > b then
   q := p and q
else
   if ( a < ( 2*b ) ) then
      p := true
   else
      q := false;

......
```

Comandos Iterativos

Suponha o comando iterativo **`while`** em **`Pascal`**:

```
 ....
while E do
    C
 ....
```

Em que E é uma expressão lógica e **`C`** são comandos.

Nessa estrutura de repetição a expressão E é avaliada, caso tenha resultado igual a **`true`**, os comandos **`C`** são executados e novamente é testada a expressão E.

A forma geral do **`while`** seria:

```
   L1:NADA
        .
        .
        .   } tradução de E
     DSVF L2
        .
        .
        .   } tradução de C
     DSVS L1
  L2:NADA
```

Comandos Iterativos

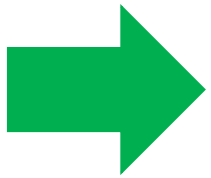
Como ficaria a tradução do trecho abaixo em **`Pascal`** para instruções da linguagem MEPA **utilizando nomes de variáveis e rótulos simbólicos**.

```
....
while ( s <= n ) do
   s := s+3*s;
......
```

Comandos Iterativos

Como ficaria a tradução do trecho abaixo em **Pascal** para instruções da linguagem MEPA **utilizando nomes de variáveis e rótulos simbólicos**.

```
....
while ( s <= n ) do
   s := s+3*s;
......
```



```
L1:NADA
   .
   .   } E
   .
   DSVF L2
   .
   .   } C
   .
   DSVS L1
L2:NADA
```

```
L1:NADA
   CRVL s
   CRVL n
   CMEG
   DSFV L2
   CRVL s
   CRCT 3
   CRVL s
   MULT
   SOMA
   ARMZ s
   DSVS L1
L2:NADA
```

Exercício

1) Considere o trecho de código abaixo, faça uma tradução do trecho utilizando **nomes de variáveis e rótulos simbólicos**.

```
while s >= 15 do
begin
     b := s + 1;
     if ( a > s ) then
         while( b > s ) do
             b := b + s
     else
         b := b + 1;
    c := a*b+s
end;
```

Comandos Iterativos

Suponha o comando iterativo `for` em **`Pascal`**:

```
....
for V of E1 to E2 do
      C
....
```

Em que **V** é um identificador, $\mathbf{E_1}$ e $\mathbf{E_2}$ são expressões aritméticas e **C** são comandos.

Nessa estrutura de repetição a variável de controle **V** é iniciada uma única vez com o valor da expressão $\mathbf{E_1}$, como se fosse o comando `V :=` $\mathbf{E_1}$.

O teste para **V <=** $\mathbf{E_2}$ é feita logo em seguida, e o incremento de **V** em uma unidade deve ser feita após ser executado dos comandos **C**.

Comandos Iterativos

A forma geral do **`for`** seria:

. } tradução a inicialização de **V** com valor de $\mathbf{E_1}$

L1:NADA

. } tradução do teste **V <=** $\mathbf{E_2}$

DSVF L2

. } tradução de **C** e incremento de **V**

DSVS L1

L2:NADA

Exercício

2) Faça a tradução do trecho abaixo em **`Pascal`** para instruções da linguagem MEPA **utilizando nomes de variáveis e rótulos simbólicos**.

```
....
for s of 1 to n do
   a := a+3*s
......
```

Tradução de programas em `Pascal` para MEPA

Considere o programa em **`Pascal`**, como ficaria a tradução do programa para a linguagem **P-código** MEPA ?

Como definir espaço de memória para as variáveis do programa ??

```
program fibonacci;
   var
      n, k:integer;
      f1, f2, f3:integer;
begin
   read(n);
   f1 := 0;
   f2 := 1;
   k  := 1;
   while k <= n do
   begin
      f3 := f1+f2;
      f1 := f2;
      f2 := f3;
      k  := k + 1
   end;
   write(n,f1)
end.
```

Tradução de programas em Pascal para MEPA

Para iniciar a tradução use a instrução `INPP`

```
INPP - Iniciar programa principal
  s <- -1
```

O programa **`EXEMPLO1`** quando for ser executado na **P-Máquina** deve reservar cinco posições iniciais na memória `M[]`, para as variáveis declaradas, para isso podemos usar a instrução `AMEM`.

```
AMEM m - ALOCAR MEMÓRIA
  s <- s+m
```

Para finalizar a execução do programa na **P-Máquina** usaremos a instrução `PARA`.

```
PARA - para execução
```

Comandos de Entrada e Saída

No programa **EXEMPLO1** temos a entrada e saída de dados realizada pelas chamadas das funções **read()** e **write()**.

read(V_1, V_2, ..., V_n)

A função lê os próximos n valores inteiros e atribui as variáveis $V_1, V_2, ..., V_n$ por exemplo:

```
read(a,b,c)
```

write(E_1, E_2, ..., E_n)

A função imprime os valores das expressões $E_1, E_2, ..., E_n$, por exemplo:

```
write(a+2,b-1,c)
```

Comandos de Entrada e Saída

Para implementar as funções de entrada e saída temos as seguintes instruções para MEPA.

```
LEIT - LEITURA
 s <- s+1
 M[s] <- "Valor digitado no teclado"
```

Exemplo:

```
read(a,b,c)
  LEIT
  ARMZ a
  LEIT
  ARMZ b
  LEIT
  ARMZ c
```

```
IMPR - IMPRESSAO
  "IMPRIMA O VALOR M[s]"
  s <- s-1
```

Exemplo:

```
write(a+2,c)
  CRVL a
  CRCT 2
  SOMA
  IMPR
  CRVL c
  IMPR
```

Tradução de programas em Pascal para MEPA

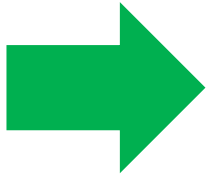
Pronto agora podemos traduzir o programa completo, **fragmentos do programa-fonte** serão usados como **comentários** a fim de tornar mais clara a tradução e também rótulos símbolos.

```
program exemplo1;
   var
      n, k:integer;
      f1, f2, f3:integer;
begin
   read(n);
   f1 := 0;
   f2 := 1;
   k  := 1;
   while k <= n do
   begin
      f3 := f1+f2;
      f1 := f2;
      f2 := f3;
      k  := k + 1
   end;
   write(n,f1)
end.
```

Exercício.

1) Após o programa traduzido, faça a execução do em MEPA e apresente o estado a memória `M[]` após execução da instrução **`f3 := f1 + f2;`** ?

2) Traduza o programa fonte ao lado para MEPA.



```
program fatorial;
var
  fat,num,cont : integer;
begin
  read(num);
  fat := 1;
  cont := 2;
  while cont <= num do
  begin
    fat := fat * cont;
    cont := cont + 1
  end;
  write(fat)
end.
```

Exercício.

3) Suponha o comando condicional `if...else`.

```
if E then
  C1
else
  C2
```

a) Escreva a função que realiza a ASDR e gera código para MEPA, considere que as funções que representam outras produções já estão implementadas.

Para realizar a tradução do comando condicional `if...else` para MEPA usaremos a função **`proximo_rotulo()`** que retorna o valor do próximo rótulo consecutivo positivo, por exemplo **`1`**, **`2`**, **`3`**, ...

Exercício.

4) Suponha o comando iterativo `while...do`.

```
while E do
    C
```

a) Escreva a função que realiza a ASDR e gera código para MEPA, considere que as funções que representam outras produções já estão implementadas.

Para realizar a tradução do comando condicional `if...else` para MEPA usaremos a função **`proximo_rotulo()`** que retorna o valor do próximo rótulo consecutivo positivo, por exemplo **`1`**, **`2`**, **`3`**, ...

Exercício.

5) Suponha o comando iterativo `do...while`.

```
do
   C1
while E1
```

Considere que o laço é executado enquanto a expressão **E1** é verdadeira e o comando **C1** é pelo menos uma vez e enquanto **E1** for verdadeira.

a) Dê um exemplo de um trecho de programa na linguagem definida no trabalho que utiliza o comando e faça a sua tradução para MEPA.

b) Apresente uma sugestão de uma produção para o comando, usando a gramática do trabalho.

c) Escreva a função que realiza a ASDR e gera código para MEPA, considere que a função **`proximo_rotulo()`** já está implementada.

Muito importante para aprender mais !!!

Capítulo 10.3 o livro **Implementação de Linguagens de Programação** do professor Tomasz Kowaltowski explica um pouco mais com gerar código, o livro está disponível em:

https://drive.google.com/file/d/1DyqeBUgayQpjh1yaHziQgm8WCE7u7ZaN/view

Grato !

Bons estudos !